

Exercice 25 — Pierre-feuille-ciseaux (version évolutive)



Comment ça marche

Contrairement aux exercices précédents, tu ne trouveras pas la solution complète ici : on va la construire ensemble, petit bout par petit bout.

À chaque étape, un petit morceau de code est déjà écrit pour toi (« l'aide au départ ») et une partie reste à compléter. Teste ton code à chaque étape avant de passer à la suivante — ne saute pas d'étape !

Étape 1 — Un seul cas : l'égalité

On commence tout petit : juste le cas où les deux joueurs jouent la même chose.

```
def jeu(a, b):
    if a == b:
        return "égalité"
    # ... à compléter dans les étapes suivantes

print(jeu("pierre", "pierre")) # doit afficher égalité
print(jeu("feuille", "feuille")) # doit afficher égalité
```

👉 Teste ce code tel quel (il ne gère que l'égalité pour l'instant). Que se passe-t-il si tu appelles `jeu("pierre", "ciseaux")` ? Pourquoi, à ton avis ?

Étape 2 — Un premier cas de victoire

La pierre casse les ciseaux. Complète la fonction avec ce premier cas, en ajoutant un `elif` :

```
def jeu(a, b):
    if a == b:
        return "égalité"
    elif a == "pierre" and b == "ciseaux":
        return "a gagne"
    # ... il manque encore plusieurs cas
```



Aide

Et si c'est `b` qui joue "pierre" et `a` qui joue "ciseaux" ? Il te faut un second `elif`, symétrique de celui-ci, qui renvoie cette fois "b gagne".

Ajoute-le toi-même avant de continuer.

Étape 3 — Tous les autres cas de victoire

Fais le tableau à la main : il y a 9 combinaisons possibles de `(a, b)`. Tu as déjà traité 3 cas (égalité, pierre bat ciseaux dans les deux sens). Il en reste 6 : la feuille contre la pierre, les ciseaux contre la feuille — dans les deux sens à chaque fois.

```
    # feuille bat pierre
    elif a == "feuille" and b == "pierre":
        return "a gagne"
    elif a == "pierre" and b == "feuille":
        return "b gagne"
```

```
# ciseaux bat feuille : à toi de compléter
elif ... :
    ...
elif ... :
    ...
```

👉 Une fois tous les cas ajoutés, ta fonction doit avoir : 1 if (égalité) + 6 elif (un par sens de victoire pour chaque paire). Pense à vérifier qu'il n'y a pas de cas « oublié » : chacune des 9 combinaisons doit renvoyer quelque chose.

Étape 4 — Vérifier systématiquement (comme à l'exercice 13 !)

Plutôt que de tester les 9 cas un par un à la main, réutilise l'idée des boucles imbriquées de l'exercice 13 pour les vérifier toutes d'un coup :

```
choix = ["pierre", "feuille", "ciseaux"]
for a in choix:
    for b in choix:
        print(a, b, "->", jeu(a, b))
```

👉 Relis les 9 lignes affichées : est-ce que chaque résultat correspond bien à ce que tu attends ? Si une ligne affiche None, c'est qu'un cas a été oublié à l'étape 3.

Étape 5 — b. Simuler une partie au hasard

Aide au départ

Le module random propose random.choice(une_liste), qui tire un élément au hasard dans une liste.

```
from random import choice

choix = ["pierre", "feuille", "ciseaux"]

def partie_aleatoire():
    a = choice(choix)
    b = choice(choix)
    resultat = jeu(a, b)
    print(a, "contre", b, "->", resultat)
    return resultat
```

👉 Appelle partie_aleatoire() plusieurs fois de suite : les résultats changent-ils bien à chaque appel ?

Étape 6 — c. Simuler 50 tours et compter les scores

Aide au départ

Il te faut une variable de score pour chaque joueur (et une pour les égalités), initialisées à 0 avant la boucle, puis mises à jour selon ce que renvoie partie_aleatoire().

```
score_a = 0
score_b = 0
egalites = 0

for tour in range(50):
    resultat = partie_aleatoire()
    if resultat == "a gagne":
```

```
        score_a = ...
    elif resultat == "b gagne":
        score_b = ...
    else:
        egalites = ...

print("Score final :", score_a, "-", score_b, "- égalités :", egalites)
```

👉 Complète les trois lignes avec « ... » pour incrémenter le bon compteur à chaque tour, puis lance 50 tours : la somme `score_a + score_b + egalites` doit toujours valoir 50.